Product Engineer Take-home Assessment: Kaizen Wheels

> 💡 Thanks for chatting with us about the Software Engineer, Product position! We’re excited to have you complete the next steps, which is a technical take-home assessment. Below, you’ll find the prompt, how long to spend, and how we’ll evaluate. Please reach out to us if you have any questions & good luck!

The Task:

You'll spend ~2 hours on a small repo that powers a fake car rental service (KaizenWheels). You'll fix a bug, build a feature, and refactor what you shipped. You'll submit your code, your AI transcripts, and a short writeup.

Two non-negotiables:

1. **Submit your full AI transcripts in `AI_LOG.md`.** No transcript, no review. We'll send it back.
2. **Time-box to 2 hours.** If you run out, write what you *would have done* — don't leave it blank.

Time budget: 2 hours

Please stick to this! We're looking for how you prioritize under constraints, not how polished you can make it in 6 hours.

If you run out of time on any section: in `DECISIONS.md`, write something like *"I ran out of time on X — here's what I would have done with another 30 minutes"* and lay out the plan. Your thinking is more valuable to us than your ability to finish. Strong candidates often earn more credit for a well-explained partial submission than a rushed complete one.

Setup

- Download the code here:

  kaizen-take-home.zip

- Stack: Next.js / TypeScript / Node / Postgres / Prisma
- Run the app locally before you start the clock. If setup is broken, let us know — we'll fix it, you don't lose time
- Commit incrementally — we read the commit history
- When you're done, send us a zip or hosted repo.

AI use — required, not optional

Use an AI coding tool of your choice (Claude, Cursor, Copilot — whatever you actually reach for). Keep the full transcript and include it in `AI_LOG.md` in your repo. Paste the conversation, link to shares, or export from your tool — any format is fine as long as it's the full thing.

We're not testing whether you can write code without AI. We're testing whether you use AI the way a senior engineer does — as a force multiplier, with judgment. That means we want to see how you prompt, what you verify, when you push back, and what you catch.

Submissions without an AI transcript will not be reviewed.

The work

Part 1 — Fix the Bug (~20 min)

Users are reporting that the price filter is broken:

> "I want to hide results above $125/hr, but I can't figure out how to do that."

> "MY BUDGET IS $100 PER HOUR BUT IT'S SHOWING ME VERY EXPENSIVE CARS???"

Find the bug. Fix it. In `DECISIONS.md`, write 2–3 sentences explaining the root cause and your fix. Bonus if you note whether the fix should be configurable.

Part 2 — Discounts (~60 min)

We'd like to add a holiday discount that satisfies these requirements:

- A reservation that includes a holiday but does not start or end on that holiday should receive a 17% discount off the total price.
- A reservation for more than 3 days should receive a $10/hr discount on the hourly rate.
- These discounts cannot both apply at the same time. If they conflict, the discount with the best price applies to the reservation.
- Visitors should see the discount reflected during search and checkout, including on the review page.

Fictional holidays to work with: Jan 21, Feb 12, Mar 04, May 02, Jun 16, Jul 26, Aug 03, Sep 01, Nov 05, Dec 18.

Part 3 — Refactor (~25 min)

Revisit the pricing/discount code you just shipped. refactor it. Clean it up. Extract what should be extracted. Name things well. Add tests if you didn't already.

In `DECISIONS.md`, add a short section explaining:

- What you changed and why
- What you'd do differently if you weren't time-boxed

Bonus — UX callout (5 min, optional)

Walk through the app briefly. Pick one UX or product issue you'd flag to a designer on your team. Two sentences in `DECISIONS.md` is enough.

What to submit

1. Link to your fork with all work committed
2. `AI_LOG.md` — your AI prompts/transcripts. **Required.**
3. `DECISIONS.md` — bug writeup, refactor rationale, any "ran out of time" plans, optional UX callout

---

What strong looks like

The bar isn't "finished everything." It's:

- Your root-cause bug writeup names *why* it broke, not just *what* you changed
- You noted at least one real tradeoff ("I chose X over Y because...")
- If you didn't finish, you wrote a specific, concrete "next 30 minutes" plan
- Your refactor in Part 3 is meaningfully cleaner — not just rearranged

---

What we don't care about

- Pixel-perfect UI — use what's there
- Tests for code you didn't touch
- Deployment, CI, or infra

Use your 2 hours on the work that actually tests judgment.

How we'll evaluate

- **AI fluency** — how you prompt, how you verify, how you push back on bad output
- **Code quality** — clean, readable, would-not-be-embarrassing-at-scale

- **Debugging** — root-cause thinking vs. patching symptoms
- **Product judgment** — edge cases, UX, what you flagged

No gotchas or hidden tests - ship what you'd actually ship on a Tuesday.

Questions

If something is ambiguous, ask before you start! Good luck!

— The Kaizen Team